Curs 1 P.C. 2

Nota finală = min {0.5 * notă bale + 0.5 * notă lucrare + 0.1 * notă seminar, 10}

Complexitatea algoritmilor

1) timp de execuție (complexitate computațională)
2) spațiul de memorie necesar

timp de execuție (complexitate computațională) → nr. aproximativ de operații elementare efectuate de algoritm în funcție de dimensiunile datelor de intrare

Notația asimptotică $n \to \infty$

$O(n^2)$ → n^2 = nr aprox de op. elem. efectuate

dim. datelor de intrare

Op. elem. →
- op. aritmetice + op. de atribuire (+ - * /)
- op. de decizie + (op. de salt)
- op. de citire / scriere

op. salt:

```
x=1;
E:
    cout << x << endl;
    x++;
    if (x <= 5) goto E;
```

Instrucțiunile repetitive sunt atomice. Nu le descompunem în operații elementare

$\Theta(n^2)$ complexitatea computațională maximă

Ex $\boxed{1}$ max. dintr-un tablou unidimensional

$$\Theta(\underset{\text{citire } n}{1} + \underset{\text{citire elem tablou}}{n} + \underset{\text{inițializare max}}{1} + \underset{\text{op. de date}}{n-1} + \underset{\text{op. de atribuire ul max}}{n-1+1})$$

afișare

```
vmax = v[0];
for (i=1; i<n; i++)
    if (v[i] > vmax)
        vmax = v[i];
```

$\Rightarrow \Theta(3n+1) \approx \Theta(n)$ (constantele nu contează, chiar dacă sunt multiplicative sau aditive)

$\Theta(5n^2+5n+7) \approx \Theta(n^2+n) \approx \Theta(n^2)$

Se păstrează elementul care contează la limite

$\Theta(2^n+n^2) \approx \Theta(2^n)$

$\Theta(n+\log_2 n) \approx (\Theta(n))$

Ex $\boxed{2}$ sortare prin interschimbare

$$\Theta(\underset{\text{citire}}{1} + \underset{\text{citire elem}}{n} + \underset{\text{ori}}{\frac{n(n+1)}{2}} * \underset{\text{op elem}}{4} + \underset{\text{afișare tablou}}{n})$$

$= \Theta(1+n+2n^2-2n+n) = \Theta(2n^2+1) \approx$

$\approx \Theta(n^2)$

```
for (i=0; i<n; i++)
    for (j=i+1; j<n; j++)
        if (v[i] > v[j]) {
        swap(v[i], v[j]);
        }
}
```

E:

Clase uzuale de complexitate (în ordine crescătoare)

1) $\Theta(1) \rightarrow$ complexitate constantă

2) $\Theta(\log_B n) \rightarrow$ complexitate logaritmică

Ex: Operația de căutare binară $\rightarrow \Theta(\log_2 n)$ $\log_2 8 = x \Rightarrow 2^3 = 8$

Algoritmul de căutare liniară $\rightarrow \Theta(n)$ $\frac{8}{2} = \frac{4}{2} = \frac{2}{2} = 1$ 3 ori

funcție program

! OBS: Complexitatea unui algoritm nu poate fi mai mică decât complexitatea citirii datelor de intrare sau scrierii datelor de ieșire!

3) $\Theta(\text{sqrt}(n))$

4) $\Theta(n) \rightarrow$ complexitate liniară

5) $\Theta(n \cdot \log_2 n) \rightarrow$ pseudologaritmică

Quicksort, Mergesort, Heapsort

C.A.R. Hoare (Nu se poate o sortare mai bună)

Analiză amortizată

sortare căutare binară

comp. medie a unei op. de căutare $= \frac{n \log_2 n + k \cdot \log_2 n}{k} = \frac{n \log n}{k} + \log_2 n$

$k \approx n$

$\log_2 n$

8) $\Theta(a^n), a \geq 2 \rightarrow$ complexitate exponențială

6) $\Theta(n^2) \rightarrow$ complexitate pătratică

7) $\Theta(n^k), k \geq 3 \rightarrow$ complexitate polinomială

9) $\Theta(n!)$